Lab 7 Instructions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | October 21, 2026

1 Problem Statement

Edge devices like the NVIDIA Jetson Nano/Orin are constrained by limited memory bus bandwidth. When capturing frames from an image sensor over CSI (Camera Serial Interface), an image undergoes several transitions before a neural network can consume it:

1. **Wire Transfer:** Unprocessed Bayer data arrives as packed bits (e.g., `raw10`).
2. **ISP Processing:** The hardware Image Signal Processor demosaics and packs data into planar or semi-planar formats (e.g., `nv12`, `i420`).
3. **Host Ingestion:** Software runtimes such as OpenCV typically convert frames to packed channels (e.g., `bgr`, `rgb`, or aligned `bgrx`).
4. **Inference Preprocessing:** The model pipeline resizes spatial dimensions, converts pixel channels to floating-point tensors, and normalizes values.

Every step reads from and writes to memory, generating bus traffic. Furthermore, a camera sensor may fail silently by producing dead black frames, uniform flat fields, or underexposed scenes.

Your Task: Implement an analysis and budgeting module that:

- Quantifies byte counts and bus bandwidth per frame format.
- Computes pixel statistics to automatically classify sensor health.
- Calculates memory traffic incurred by format conversion, spatial resizing, and float normalization.

2 Reference Data and Constants

The module relies on the following pixel weight factors (bpp):
Thresholds for classification:

- BLACK_MAX $= \frac{1}{255} \approx 0.00392$: Maximum intensity threshold for a completely dead/covered sensor.
- FLAT_SPREAD $= 0.02$: Difference between maximum and minimum value below which an image is considered a featureless field.
- DARK_MEAN $= 0.10$: Upper bound on mean intensity for a poorly illuminated scene.

Format Key	Bytes Per Pixel (bpp)	Architectural Description
`raw10`	1.25	10 bits per pixel packed on the CSI wire (4 px in 5 bytes)
`gray8`	1.0	8-bit luma channel only
`nv12`	1.5	Full luma plane (Y), interleaved half-resolution chroma (UV)
`i420`	1.5	Full luma plane (Y), two separate planar chroma (U and V)
`rgb`	3.0	3 independent 8-bit channels (R, G, B)
`bgr`	3.0	3 independent 8-bit channels ordered for OpenCV (B, G, R)
`bgrx`	4.0	3 image channels plus 1 unused 8-bit alignment padding byte

Table 1: Supported frame pixel formats.

3 Step-by-Step Implementation Guide

3.1 `frame_bytes(width, height, fmt)`

Purpose: Computes the exact integer number of bytes required to store an uncompressed 2D frame.

Required Inputs:

- `width` (`int`): Horizontal resolution in pixels.
- `height` (`int`): Vertical resolution in pixels.
- `fmt` (`str`): Format identifier (e.g., `"nv12"`, `"rgb"`).

Required Output:

- `int`: Total footprint in bytes.

Step-by-Step Instructions:

1. Validate that `width` and `height` are strictly positive integers (> 0). Raise a `FrameError` if either is ≤ 0 or non-integer.
2. Look up the format factor from the table using `fmt.lower()`. Raise a `FrameError` if the format is unsupported.
3. Compute raw continuous bytes: total = width $\times$ height $\times$ bpp.
4. Because hardware buses cannot allocate partial bytes, apply ceiling semantics:

$$\text{bytes} = \lceil \text{width} \times \text{height} \times \text{bpp} \rceil$$

3.2 `bytes_per_second(width, height, fmt, fps)`

Purpose: Measures the continuous memory transfer rate required to ingest a video stream.

Required Inputs:

- `width` (`int`): Horizontal resolution.
- `height` (`int`): Vertical resolution.
- `fmt` (`str`): Target format identifier.
- `fps` (`float`): Framerate in frames per second.

Required Output:

- `float`: Data throughput in bytes per second.

Step-by-Step Instructions:

1. Validate that fps > 0. Raise a `FrameError` if it is zero or negative.
2. Calculate the single-frame byte size by invoking `frame_bytes(width, height, fmt)`.
3. Multiply the single-frame size by the framerate:

$$\text{Bandwidth} = \text{frame_bytes} \times \text{fps}$$

3.3 `frame_stats(pixels)`

Purpose: Derives primary statistical moments across a collection of normalized pixel values ($v \in [0.0, 1.0]$).

Required Inputs:

- `pixels` (`Sequence[float]` or `Iterable[float]`): Linear collection of normalized scalar intensities.

Required Output:

- `dict[str, float]`: A dictionary containing:
  - `"min"`: Lowest observed intensity value.
  - `"max"`: Highest observed intensity value.
  - `"mean"`: Arithmetic mean (μ).
  - `"spread"`: Dynamic range (max − min).
  - `"sd"`: Population standard deviation (σ).
  - `"samples"`: Total sample count (N).

Step-by-Step Instructions:

1. Ingest the values into a list of floats. If empty, raise a `FrameError` (a failed read is distinct from an empty or black frame).
2. Extract minimum ($v_{\min}$) and maximum ($v_{\max}$).
3. Compute sample count $N = \text{len}(values)$ and arithmetic mean:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} v_i$$

4. Compute the population variance using all samples in the frame:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (v_i - \mu)^2$$

5. Derive standard deviation $\sigma = \sqrt{\sigma^2}$ and spread $= v_{\max} - v_{\min}$.
6. Return the values packaged into the specified dictionary structure.

3.4 `classify(stats)`

Purpose: Inspects computed metrics to determine if the capture is valid or anomalous.

Required Inputs:

- `stats` (`dict[str, float]`): Dictionary containing `"max"`, `"spread"`, and `"mean"`.

Required Output:

- `tuple[str, str]`: `(category, rationale)`, where `category` is one of `"black"`, `"flat"`, `"dark"`, or `"scene"`.

Step-by-Step Instructions:

1. Ensure keys `"max"`, `"spread"`, and `"mean"` exist in the input dictionary.
2. **Black Evaluation:** If max $\leq$ BLACK_MAX, return category `"black"`. The sensor is registering zero photon response (e.g., lens cap attached).
3. **Flat Evaluation:** If spread $<$ FLAT_SPREAD, return category `"flat"`. Contrast is absent across all pixels (e.g., disconnected or washed-out sensor).
4. **Dark Evaluation:** If mean < 0.10, return category `"dark"`. Structure exists, but exposure is insufficient.
5. **Nominal Evaluation:** Otherwise, return category `"scene"`. The image possesses proper exposure and dynamic range.

3.5 `conversion_bytes(width, height, src, dst)`

Purpose: Prices the aggregate bus traffic generated by an in-memory pixel format conversion.

Required Inputs:

- `width` (`int`): Frame width in pixels.
- `height` (`int`): Frame height in pixels.
- `src` (`str`): Input format.
- `dst` (`str`): Target output format.

Required Output:

- `int`: Total bytes transferred across the bus (reads + writes).

Step-by-Step Instructions:

1. Compare `src.lower()` with `dst.lower()`. If identical, no transformation occurs; return 0.
2. Calculate read volume: frame_bytes(width, height, src).
3. Calculate write volume: frame_bytes(width, height, dst).
4. Sum the read and write volumes:

$$\text{Traffic} = \text{Bytes}_{\text{read}} + \text{Bytes}_{\text{write}}$$

3.6 `resize_bytes(src_w, src_h, dst_w, dst_h, fmt)`

Purpose: Calculates the read/write bus transactions required to bilinearly downsample or upsample an image.

Required Inputs:

- `src_w`, `src_h` (`int`): Dimensions of the source frame.
- `dst_w`, `dst_h` (`int`): Dimensions of the destination frame.
- `fmt` (`str`): Format of the image.

Required Output:

- `int`: Sum of read bytes and written bytes.

Step-by-Step Instructions:

1. Compute the source frame size using `frame_bytes(src_w, src_h, fmt)`.
2. Compute the destination frame size using `frame_bytes(dst_w, dst_h, fmt)`.
3. Return the combined sum of both buffers.

3.7 `normalise_bytes(width, height, channels, out_bytes)`

Purpose: Computes the memory cost to convert integer color channels (typically `uint8`, 1 byte) into model input tensors (typically `float32`, 4 bytes).

Required Inputs:

- `width` (`int`): Tensor spatial width.
- `height` (`int`): Tensor spatial height.
- `channels` (`int`, default: 3): Number of color channels.
- `out_bytes` (`int`, default: 4): Number of bytes per output scalar (e.g., 4 for `float32`, 2 for `float16`).

Required Output:

- `int`: Total bus bytes required to read integer source data and write float destination tensors.

Step-by-Step Instructions:

1. Validate that width > 0, height > 0, and channels > 0. Raise a `FrameError` otherwise.
2. Calculate the total number of scalar values:

$$P = \text{width} \times \text{height} \times \text{channels}$$

3. Calculate input read memory assuming 8-bit source data (1 byte/scalar):

$$\text{Bytes}_{\text{in}} = P \times 1$$

4. Calculate output write memory:

$$\text{Bytes}_{\text{out}} = P \times \text{out_bytes}$$

5. Return $\text{Bytes}_{\text{in}} + \text{Bytes}_{\text{out}}$.

3.8 `summarise_formats(width, height)`

Purpose: Produces a comparative breakdown of all supported formats relative to canonical uncompressed `rgb`.

Required Inputs:

- `width` (`int`): Frame width.
- `height` (`int`): Frame height.

Required Output:

- `list[dict[str, Any]]`: List of dictionaries, each describing a format with:
  - `"format"`: Format name string.
  - `"bytes_per_pixel"`: Float scalar bpp.
  - `"bytes"`: Total frame byte footprint.
  - `"relative_to_rgb"`: Fractional size compared to 24-bit RGB (bytes/$\text{bytes}_{\text{rgb}}$).
  - `"note"`: Architectural note describing channel layouts.

Step-by-Step Instructions:

1. Compute baseline byte footprint: base = frame_bytes(width, height, "rgb").
2. Iterate through every identifier in `FORMATS`:
   a) Calculate N = frame_bytes(width, height, fmt).
   b) Look up bpp = _bytes_per_pixel(fmt).
   c) Calculate ratio: ratio = $\frac{N}{\text{base}}$.
   d) Retrieve the format note string from `FORMAT_NOTES`.
   e) Construct and append the dictionary to the output list.
3. Return the populated list of rows.